

Aim: Implement the Backend of compiler

To implement a simple compiler back-end simulation in C that takes Three Address Code (3AC) as input and generates equivalent Assembly-like instructions as output.

Algorithm

1. Start the program and ask the user to input the number of 3AC statements.
2. For each statement:
 - Read the statement in the form: `lhs = op1 operator op2`.
 - Parse the left-hand side (`lhs`), operand 1 (`op1`), operator, and operand 2 (`op2`).
 - Based on the operator:
 - `+` → Generate `MOV` and `ADD`
 - `-` → Generate `MOV` and `SUB`
 - `*` → Generate `MOV` and `MUL`
 - `/` → Generate `MOV` and `DIV`
 - After the operation, store the result back into the `lhs` variable.
3. Continue this for all lines and print the equivalent assembly instructions.

C Program Code

```
#include <stdio.h>
#include <string.h>

int regCount = 1;

void processLine(char line[]) {
    char lhs[10], op1[10], op[3], op2[10];

    int parsed = sscanf(line, "%s = %s %s %s", lhs, op1, op, op2);
    if (parsed != 4) {
        printf("// Unsupported or invalid format: %s\n", line);
        return;
    }

    if (strcmp(op, "+") == 0)
        printf("MOV R%d, %s\nADD R%d, %s\n", regCount, op1, regCount, op2);
    else if (strcmp(op, "-") == 0)
        printf("MOV R%d, %s\nSUB R%d, %s\n", regCount, op1, regCount, op2);
    else if (strcmp(op, "*") == 0)
```

```

        printf("MOV R%d, %s\nMUL R%d, %s\n", regCount, op1, regCount, op2);
    else if (strcmp(op, "/") == 0)
        printf("MOV R%d, %s\nDIV R%d, %s\n", regCount, op1, regCount, op2);
    else {
        printf("// Unsupported operation: %s\n", op);
        return;
    }

    printf("MOV %s, R%d\n", lhs, regCount);
    regCount++;
}

int main() {
    int n;
    char line[100];

    printf("Enter number of 3AC statements: ");
    scanf("%d", &n);
    getchar(); // consume newline

    printf("Enter 3AC statements:\n");
    for (int i = 0; i < n; i++) {
        do {
            fgets(line, sizeof(line), stdin);
        } while (line[0] == '\n');

        line[strcspn(line, "\n")] = '\0'; // remove newline
        processLine(line);
    }

    return 0;
}

```

Sample Input

```

Enter number of 3AC statements: 3
t1 = a + b
t2 = t1 * c
t3 = t2 - d

```

Sample Output

```

MOV R1, a
ADD R1, b

```

```
MOV t1, R1
MOV R2, t1
MUL R2, c
MOV t2, R2
MOV R3, t2
SUB R3, d
MOV t3, R3
```